

Maratona de Revisão: Lógica e Programação

25 Questões Práticas: De Operadores Básicos a Funções

Prof. Reginaldo Fernandes
Lógica de Programação

Instituto Federal do Ceará – Campus Tauá
Curso Técnico em Redes de Computadores

16 de setembro de 2026

Objetivo Pedagógico

Consolidar a capacidade analítica e algorítmica por meio de **25 desafios práticos progressivos**.

Distribuição dos Tópicos:

- **Parte 1 (Q01-Q05):** Operadores Matemáticos, Relacionais e Lógicos.
- **Parte 2 (Q06-Q10):** Estruturas de Decisão (if-elif-else).
- **Parte 3 (Q11-Q15):** Laços de Repetição e Vetores.
- **Parte 4 (Q16-Q25):** Funções (10 questões cobrindo todos os tipos).

Estrutura de Cada Slide:

- **Quesito:** Enunciado claro do problema computacional e especificação de entrada/saída.
- **Ajuda de Resolução:** Guia de raciocínio, dicas de sintaxe e operadores recomendados, **sem entregar o código final**.

Sumário da Apresentação

- 1 Parte 1: Programação Estruturada e Operadores
- 2 Parte 2: Estruturas de Decisão
- 3 Parte 3: Estruturas de Repetição e Vetores
- 4 Parte 4: Funções e Modularização
- 5 Resumo e Orientações Finais

Parte 1: Programação Estruturada

Operadores Aritméticos, Relacionais e Lógicos

Questões 01 a 05

Conceito Central

A programação estruturada baseia-se no fluxo sequencial e na manipulação de dados na memória através de expressões matemáticas e booleanas.

Questão 01: Operadores Aritméticos Básicos

Quesito: Custo de Deslocamento e Consumo

Desenvolva um programa que receba três valores:

- 1 Distância total da viagem em quilômetros (km);
- 2 Consumo médio do veículo em quilômetros por litro (km/l);
- 3 Preço do litro de combustível em reais (R\$).

O programa deve calcular e exibir:

- O total de litros consumidos (Distância/Consumo);
- O custo total em reais formatado com 2 casas decimais.

Ajuda de Resolução (Como Pensar)

- Converta os dados lidos do teclado para float.
- Calcule os litros com o operador de divisão (/).
- Obtenha o custo multiplicando litros pelo preço unitário (*).
- Exiba o resultado com `f"{custo:.2f}"`.

Questão 02: Divisão Inteira e Operador Módulo

Quesito: Conversão de Segundos para H:M:S

Um monitor de infraestrutura registra o tempo de atividade de um servidor em segundos inteiros.

Escreva um programa que leia esse valor inteiro e o decomponha no formato: **Horas**, **Minutos** e **Segundos restantes**.

Ajuda de Resolução (Como Pensar)

- Lembre-se: $1 \text{ h} = 3600 \text{ s}$ e $1 \text{ min} = 60 \text{ s}$.
- Calcule as horas completas com divisão inteira: $\text{horas} = \text{total} // 3600$.
- Extraia o que sobrou com o operador resto (%): $\text{resto} = \text{total} \% 3600$.
- Obtenha os minutos com $\text{minutos} = \text{resto} // 60$ e os segundos finais com $\text{segundos} = \text{resto} \% 60$.

Questão 03: Operadores Relacionais (Comparação)

Quesito: Monitor de Capacidade de Link de Rede

Um link opera a 1000 Mbps. Leia o tráfego atual (Mbps) e gere 3 variáveis booleanas (True/False), sem usar if:

- **Uso Seguro:** Tráfego estritamente menor que 80% da capacidade;
- **Atenção Crítica:** Tráfego atingiu ou superou 90% da capacidade;
- **Sobrecarga:** Tráfego estritamente superior a 100% da capacidade.

Exiba o resultado booleano de cada uma das três avaliações.

Ajuda de Resolução (Como Pensar)

- Operadores de comparação ($<$, $>=$, $>$) retornam diretamente True ou False.
- Calcule os limites multiplicando a capacidade por 0.80, 0.90 e 1.0.
- Guarde cada comparação direto em uma variável booleana e a imprima.

Questão 04: Operadores Lógicos Comportamentais

Quesito: Controle de Acesso ao Datacenter

A catraca eletrônica deve autorizar a abertura da porta quando:

- O usuário possui credencial (True/False) **E** o horário atual está dentro da janela permitida (8h às 18h);
- **OU** o alarme de emergência está ativado (True/False).

Construa a expressão lógica em Python que determine se a porta deve destravar.

Ajuda de Resolução (Como Pensar)

- Identifique: `tem_cricha` (bool), `hora` (int) e `emergencia` (bool).
- A janela de horário é: `(hora >= 8 and hora <= 18)`.
- Conecte as condições com os operadores `and` e `or`.
- Use parênteses para garantir que o `or` avalie a emergência como exceção geral.

Questão 05: Expressões Aritmético-Lógicas

Quesito: Verificador de Regra de Ano Bissexto

Um ano é bissexto se atender às regras do calendário gregoriano:

- É divisível por 4 e não é divisível por 100; **OU**
- É divisível por 400.

Solicite um ano inteiro e avalie se ele é bissexto usando **uma única expressão booleana** (sem `if/else`). Imprima apenas o resultado booleano.

Ajuda de Resolução (Como Pensar)

- Divisibilidade por N é avaliada com operador de resto: `ano % N == 0`.
- Não ser divisível por 100: `ano % 100 != 0`.
- Combine as condições em uma só linha usando operadores `and` e `or` com parênteses adequados.

Parte 2: Estruturas de Decisão

Desvios Condicionais Simples, Compostos e Encadeados

Questões 06 a 10

Conceito Central

As estruturas de decisão permitem que o fluxo de execução tome caminhos distintos a depender da avaliação de condições lógicas em tempo de execução.

Questão 06: Decisão Simples e Composta

Quesito: Radar de Trânsito Inteligente

Leia a velocidade registrada de um veículo em via com limite de 60 km/h.

- Até 60 km/h: exiba "Velocidade permitida".
- Acima de 60 km/h: informe "Multado!", calcule e exiba a multa a R\$ 7,00 por km/h excedente.

Ajuda de Resolução (Como Pensar)

- Estruture com if-else testando: $\text{velocidade} > 60$.
- No bloco do if, calcule o excesso: $\text{excesso} = \text{velocidade} - 60$.
- Multiplique o excesso por 7.00 para obter o valor total da penalidade.

Questão 07: Decisão Encadeada (if-elif-else)

Quesito: Triagem de Glicemia em Jejum

Receba a glicose no sangue de um paciente (mg/dL) e classifique:

- Menor que 70 mg/dL: "**Hipoglicemia**"
- De 70 a 99 mg/dL: "**Normal**"
- De 100 a 125 mg/dL: "**Pré-diabetes**"
- 126 mg/dL ou mais: "**Diabetes Estabelecido**"

Ajuda de Resolução (Como Pensar)

- Use uma cadeia ordenada de if-elif-else do menor para o maior.
- Se o primeiro teste `glicose < 70` falhar, o próximo ramo já sabe que o valor é ≥ 70 , bastando testar `elif glicose <= 99`.

Questão 08: Condições Múltiplas e Geometria

Quesito: Validação e Classificação de Triângulos

Receba três lados A , B e C . O programa deve:

- 1 Verificar se formam triângulo (cada lado menor que a soma dos outros dois);
- 2 Se formarem, classificar: **Equilátero** (3 iguais), **Isósceles** (2 iguais) ou **Escaleno** (todos diferentes);
- 3 Caso contrário, avisar que não formam um triângulo.

Ajuda de Resolução (Como Pensar)

- Valide a existência com $(A < B + C)$ and $(B < A + C)$ and $(C < A + B)$.
- Aninhe a classificação dentro do bloco verdadeiro:
 - Teste 3 lados iguais: $A == B == C$ (Equilátero).
 - Teste 2 lados iguais: $(A == B)$ or $(B == C)$ or $(A == C)$ (Isósceles).
 - O restante cai no `else` (Escaleno).

Questão 09: Tabela Progressiva de Descontos

Quesito: Política Comercial de Descontos

Um e-commerce aplica descontos conforme o valor bruto da compra:

- Até R\$ 100,00: 0% — De R\$ 100,01 a R\$ 300,00: 10%
- De R\$ 300,01 a R\$ 500,00: 15% — Acima de R\$ 500,00: 20%

Exiba: taxa percentual, valor descontado em R\$ e valor final a pagar.

Ajuda de Resolução (Como Pensar)

- Defina uma variável `taxa` e use `if-elif-else` apenas para ajustá-la (0.0, 0.10, 0.15 ou 0.20).
- Fora do bloco condicional, faça os cálculos apenas uma vez:
 $\text{desconto} = \text{valor} \times \text{taxa}$ e $\text{total} = \text{valor} - \text{desconto}$.

Questão 10: Menu de Operações e Prevenção de Falhas

Quesito: Mini Calculadora com Salvaguarda

Apresente um menu (1-Soma, 2-Subtração, 3-Multiplicação, 4-Divisão), receba dois números e execute a opção escolhida.

- Na divisão, evite falha no programa impedindo a divisão por zero.
- Para opções inexistentes, exiba "Opção Inválida".

Ajuda de Resolução (Como Pensar)

- Estruture as escolhas usando `if-elif-else`.
- Na opção de divisão, verifique se o segundo número é diferente de zero (`num2 != 0`) antes de calcular.
- Use o `else` final para tratar qualquer opção digitada fora do intervalo 1 a 4.

Parte 3: Repetição e Vetores

Laços de Repetição (for, while) e Manipulação de Listas

Questões 11 a 15

Conceito Central

Laços automatizam o processamento sequencial de dados, viabilizando cálculos estatísticos, buscas lineares e filtros em coleções.

Questão 11: Laço Contado e Vetor Acumulador

Quesito: Análise Térmica Semanal

Leia a temperatura máxima dos 7 dias da semana e guarde em uma lista. Ao final, determine e mostre:

- 1 A média aritmética simples das temperaturas;
- 2 A maior temperatura registrada;
- 3 Quantos dias tiveram temperatura estritamente superior à média.

Ajuda de Resolução (Como Pensar)

- Use um laço `for` para coletar as 7 entradas com `.append()`.
- Obtenha a média com `soma/7` (usando `sum()` ou acumulador).
- Em um segundo laço, conte quantos valores na lista são maiores que a média calculada usando uma condição `if temp > media:`.

Questão 12: Repetição com Sentinela (`while`)

Quesito: Totalizador de Terminal de Saque

Simule o fechamento de um caixa de autoatendimento bancário:

- Leia saques contínuos até que o usuário digite 0 (parada).
- Rejeite valores negativos com aviso e desconsidere-os da contagem.
- Exiba o total de saques válidos efetuados e a soma total sacada.

Ajuda de Resolução (Como Pensar)

- Use um laço `while True`: com leitura do valor a cada volta.
- Se o valor for 0, encerre a repetição com `break`.
- Se for negativo, alerte o usuário e volte ao topo com `continue`.
- Caso contrário, some o saque ao total e incremente o contador.

Questão 13: Varredura de Vetor e Filtragem

Quesito: Separação de Números Pares e Ímpares

Leia 10 números inteiros para uma lista inicial. Em seguida:

- Crie uma lista com os números pares e outra com os ímpares;
- Exiba o tamanho e os elementos contidos nas três listas.

Ajuda de Resolução (Como Pensar)

- Crie duas listas vazias: `pares = []` e `impares = []`.
- Itere na lista com `for n in lista:`.
- Verifique a paridade com `n % 2 == 0`: adicione em `pares` ou em `impares` com `.append()`.
- Exiba o número de itens com `len()`.

Questão 14: Busca Linear em Vetores com Flag

Quesito: Verificador de Matrícula em Lista de Acesso

Dada uma lista com 8 matrículas pré-cadastradas, leia uma matrícula informada pelo usuário e faça a busca:

- Se presente, mostre: "Autorizado na posição X".
- Se ausente, mostre: "Acesso Negado: Matrícula não cadastrada".

Ajuda de Resolução (Como Pensar)

- Inicie uma flag booleana de controle: `achou = False`.
- Percorra os índices com `for i in range(len(lista)):`.
- Se o item do índice for igual à busca, marque `achou = True`, exiba a posição `i` e interrompa com `break`.
- Ao final do laço, teste: `se not achou:`, avise que não encontrou.

Questão 15: Inversão Algorítmica de Vetor

Quesito: Espelhamento de Elementos sem Métodos Prontos

Preencha uma lista com 6 números fornecidos pelo usuário.

Crie uma nova lista com os mesmos elementos em **ordem invertida**, sem usar `.reverse()` ou fatiamento `[::-1]`.

Ajuda de Resolução (Como Pensar)

- O último elemento está em tamanho $- 1$ e o primeiro em 0 .
- Crie uma nova lista vazia para armazenar o resultado.
- Percorra a lista de trás para frente usando:
`range(len(lista) - 1, -1, -1)`.
- Insira cada elemento lido na nova lista com `.append()`.

Parte 4: Modularização com Funções

Parâmetros, Retornos e Escopo de Variáveis

Questões 16 a 25 (10 Desafios Didáticos)

Conceito Central

Funções dividem problemas complexos em sub-rotinas reutilizáveis. Variam em dois eixos: se recebem entradas (**parâmetros**) e se devolvem respostas (**return**).

Questão 16: Função Sem Parâmetro e Sem Retorno

Quesito: Cabeçalho Padrão Institucional

Crie a função `exibir_cabecalho()` que imprima na tela um banner formatado com:

- Instituição: **IFCE Campus Tauá**;
- Curso: **Técnico em Redes de Computadores**;
- Molduras decorativas com o caractere "=".

Invoque essa função no início do programa principal.

Ajuda de Resolução (Como Pensar)

- Declare com `def exibir_cabecalho():` (parênteses vazios).
- Não utilize a instrução `return`; utilize comandos `print()`.
- Chame a função no código principal apenas escrevendo: `exibir_cabecalho()`.

Questão 17: Função Sem Parâmetro e Sem Retorno

Quesito: Menu de Ajuda ao Operador

Implemente a função `exibir_menu_ajuda()` que mostre opções de teclas de atalho de um terminal (`[S]` Salvar, `[C]` Carregar, `[Q]` Sair). O programa principal deve chamar a função quando o usuário digitar "ajuda".

Ajuda de Resolução (Como Pensar)

- Escreva a rotina contendo apenas saídas de tela organizadas.
- No fluxo principal, compare a entrada lida: `if opcao == "ajuda":`.
- Dentro do `if`, invoque a função pelo seu nome.

Questão 18: Função Com Parâmetro e Sem Retorno

Quesito: Notificador de Serviços de Infraestrutura

Crie a função `notificar_servico(nome, porta, status)` onde `status` é booleano. A função deve exibir:

- Se True: "[OK] Serviço: <nome> | Porta: <porta> -> ATIVO".
- Se False: "[FALHA] Serviço: <nome> | Porta: <porta> -> PARADO".

Ajuda de Resolução (Como Pensar)

- Receba 3 argumentos: `def notificar_servico(nome, porta, status):`.
- Avalie o parâmetro booleano com `if status: ... else: ....`
- Exiba a mensagem formatada com f-string sem usar `return`.

Questão 19: Função Com Parâmetro e Sem Retorno

Quesito: Divisor Visual Parametrizado

Crie a função `desenhar_separador(simbolo, comprimento)` que receba um caractere textual e a quantidade de repetições horizontais. A função deve imprimir a linha gerada no console.

Ajuda de Resolução (Como Pensar)

- Em Python, multiplicar string por número repete o texto: `simbolo * comprimento`.
- Imprima a string resultante diretamente na tela.
- Teste com diferentes argumentos, como: `desenhar_separador("-", 30)`.

Questão 20: Função Sem Parâmetro e Com Retorno

Quesito: Leitor com Validação de Entrada Estrita

Implemente a função `ler_inteiro_positivo()` sem parâmetros.

- Solicite continuamente um número inteiro até que seja maior que zero;
- Assim que receber um valor estritamente positivo, devolva-o com `return`.

Ajuda de Resolução (Como Pensar)

- Use um laço `while True`: dentro da função.
- Leia o número com `int(input())`.
- Se `valor > 0`, execute `return valor`.
- O `return` encerra o laço e a função no mesmo instante.

Questão 21: Função Sem Parâmetro e Com Retorno

Quesito: Gerador de Código de Autenticação (OTP)

Escreva a função `gerar_token_acesso()` sem parâmetros que sorteie e retorne um código de 6 dígitos inteiros (entre 100000 e 999999), simulando um token de segurança de dois fatores.

Ajuda de Resolução (Como Pensar)

- Utilize o módulo `random` do Python.
- Use a função `random.randint(100000, 999999)`.
- Retorne o valor sorteado com a instrução `return`.
- No programa principal, guarde o retorno em uma variável.

Questão 22: Função Com Parâmetro e Com Retorno

Quesito: Conversor Multi-Escalas de Temperatura

Crie a função `converter_celsius(temp_c, escala)` onde `escala` é "F" ou "K":

- Fórmulas: $F = C \times 1.8 + 32$ e $K = C + 273.15$.
- Calcule e retorne o resultado numérico; se a escala for inválida, retorne `None`.

Ajuda de Resolução (Como Pensar)

- Use `escala.upper()` para aceitar maiúsculas e minúsculas.
- Estruture um `if-elif-else` para checar "F" e "K".
- Calcule a conversão e faça `return` do valor obtido.

Questão 23: Função Com Parâmetro e Com Retorno

Quesito: Predicado Booleano de Número Primo

Crie a função `eh_primo(numero)` que receba um inteiro positivo e retorne `True` se for primo, ou `False` caso não seja.

Ajuda de Resolução (Como Pensar)

- Se o número for ≤ 1 , retorne imediatamente `False`.
- Faça um laço `for d in range(2, int(numero**0.5) + 1):`.
- Se encontrar divisor com resto zero (`numero % d == 0`), retorne `False`.
- Se o laço terminar sem divisores, retorne `True`.

Questão 24: Função Com Parâmetro e Retorno Múltiplo

Quesito: Localizador de Extremos e Posição em Vetor

Crie a função `analisar_pico(leituras)` que receba uma lista de números reais. A função deve retornar dois valores simultaneamente: o maior valor encontrado e o índice da primeira ocorrência desse maior valor.

Ajuda de Resolução (Como Pensar)

- Inicialize `maior = leituras[0]` e `posicao = 0`.
- Percorra a lista com índices usando `for i in range(1, len(leituras)):`.
- Se `leituras[i] > maior`, atualize ambas as variáveis.
- Devolva as duas variáveis com `return maior, posicao`.

Questão 25: Função Com Parâmetro e Com Retorno de Vetor

Quesito: Filtro e Higienização de Conjunto de Dados

Crie a função `filtrar_acima_limiar(valores, limiar)` que receba uma lista e um número de corte. Retorne uma **nova lista** apenas com os elementos estritamente maiores que o limiar.

Ajuda de Resolução (Como Pensar)

- Inicialize uma lista vazia: `resultado = []`.
- Itere sobre os itens com `for item in valores:`.
- Se `item > limiar`:, adicione à nova lista com `resultado.append(item)`.
- Finalize retornando a nova lista: `return resultado`.

Matriz de Decisão: As Quatro Formas de Funções

Classificação	Entrada	Saída	Exemplo Típico
Sem Parâm. e Sem Retorno	Não	Não	Banners e menus no console
Com Parâm. e Sem Retorno	Sim	Não	Mensagens de log e alertas
Sem Parâm. e Com Retorno	Não	Sim	Leituras com validação, tokens
Com Parâm. e Com Retorno	Sim	Sim	Cálculos, filtros e buscas

Tabela: Matriz dos Quatro Tipos de Funções em Programação.

Regra Fundamental

- **Entrada:** Recebida pelos parâmetros entre os parênteses (...).
- **Saída:** Devolvida ao código chamador por meio da instrução `return`.

Checklist para Resolução de Algoritmos

1. Análise Prévia

- Identifique as entradas necessárias.
- Defina as saídas esperadas.
- Escolha os tipos (int, float, list).

2. Construção e Teste

- Isole a lógica em blocos ou funções.
- Realize o teste de mesa no papel.
- Teste valores de borda (0, negativos).

"Primeiro resolva o problema. Depois escreva o código."

Excelente trabalho na revisão!

Pratique a implementação de cada um dos 25 quesitos.

Prof. Reginaldo Fernandes
reginaldo.fernandes@ifce.edu.br

IFCE Campus Tauá
Curso Técnico em Redes de Computadores